

Build a Database App for Your IA

IB Computer Science · using the class Flask + SQLite template (“ishflask”)

This guide shows you how to turn the class database template into your own database-backed app for your Internal Assessment. The template is a safe, working starting point — your job is to make it yours.

Do you even need this? If your IA is a simple tool, plain Python + sqlite3 (no web app) is a perfectly good IA and is simpler. Use this template only if you want a web app / API — a program that answers requests with data.

What counts as YOUR work: Your IA must be your own programming. The template is just a skeleton. You must supply your own database, your own routes, and your own SQL. Do not submit the medical example.

Step 1 — Design your database

- Open DB Browser for SQLite.
- Create your own tables, fields and relationships for your topic (a gym, a library, a booking system...).
- Save it as `mydata.db`.
- Tip: you can also build it in code — see `seed_medical.py` in the template.

Step 2 — Get the template

```
git clone https://github.com/tanasel/ishflask.git my-ia
cd my-ia
```

Then copy your `mydata.db` into the `databases/` folder.

Step 3 — Write your own routes (this is your IA work)

Open `app.py`, delete the example medical routes, and write routes for your own tables.

Example for a `Members` table:

```
@app.get("/members")
def members():
    return jsonify(query_db("databases/mydata.db",
                           "SELECT * FROM Members"))
```

Add the queries your project needs: searches, filters, and JOINS across your tables.

Step 4 — Let your app add and change data

The template only *reads* data (it is read-only on purpose, for safety). A real app usually needs to add, edit and delete records — this is where you show `INSERT` / `UPDATE` / `DELETE` (syllabus A3.3.3):

```
import sqlite3

def write_db(sql, args=()):
    conn = sqlite3.connect("databases/mydata.db") # not read-only
    conn.execute(sql, args)
```

```

conn.commit()
conn.close()

@app.post("/members")
def add_member():
    d = request.get_json()
    write_db("INSERT INTO Members (Name, Email) VALUES (?, ?)",
            (d["name"], d["email"]))
    return jsonify({"status": "added"})

```

Stay safe from SQL injection: Always use ? placeholders for values — never paste user input straight into an SQL string.

Step 5 — Run it for your video

```

python3 -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install -r requirements.txt
flask --app app run                # open http://127.0.0.1:5000

```

You do not need to host it online — running on your own laptop is fine for the 5-minute video.

Your checklist — “is this really mine?”

- ☐ My own database design (tables, fields, relationships)
- ☐ My own routes in app.py
- ☐ My own SQL queries, including at least one JOIN
- ☐ Add / edit / delete works — not just read
- ☐ I removed the medical example
- ☐ (Optional) my own web page or front end

Safety & getting help

- Use made-up / sample data only. Never put real personal data in a database you publish online — anyone could read it.
- The template’s `README.md` and `client_example.py` show working examples.
- Project repo: github.com/tanasel/ishflask — and ask your teacher if you get stuck.